day_2_Debarshi

Debarshi Das Adhikary

2026-07-24

Define the directory path containing our CEL files

```
celpath <- "/home/ibab/bio-env"
```

Load packages for microarray processing

```
library(affy)
```

```
## Loading required package: BiocGenerics
```

```
## Loading required package: generics
```

```
##
## Attaching package: 'generics'
```

```
## The following objects are masked from 'package:base':
##
##     as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
##     setequal, union
```

```
##
## Attaching package: 'BiocGenerics'
```

```
## The following objects are masked from 'package:stats':
##
##     IQR, mad, sd, var, xtabs
```

```
## The following objects are masked from 'package:base':
##
##     anyDuplicated, aperm, append, as.data.frame, basename, cbind,
##     colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
##     get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
##     order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
##     rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
##     unsplit, which.max, which.min
```

```
## Loading required package: Biobase
```

```
## Welcome to Bioconductor
##
##     Vignettes contain introductory material; view with
##     'browseVignettes()'. To cite Bioconductor, see
##     'citation("Biobase")', and for packages 'citation("pkgname")'.
```

```
## Error in assign("disableThreading", function() { :
##   cannot add bindings to a locked environment
## Error in assign("disableThreading", function() { :
##   cannot add bindings to a locked environment
```

```
library(limma)
```

```
##
## Attaching package: 'limma'
```

```
## The following object is masked from 'package:BiocGenerics':
##
##     plotMA
```

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.2.1     ✔ readr     2.2.0
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ ggplot2   4.0.3     ✔ tibble    3.3.1
## ✔ lubridate 1.9.5     ✔ tidyr     1.3.2
## ✔ purrr     1.2.2
```

```
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::combine()    masks Biobase::combine(), BiocGenerics::combine()
## ✖ dplyr::filter()     masks stats::filter()
## ✖ dplyr::lag()        masks stats::lag()
## ✖ lubridate::pm()     masks affy::pm()
## ✖ ggplot2::Position() masks BiocGenerics::Position(), base::Position()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

Read raw microarray data from the specified directory

```
yeast_microchip <- ReadAffy(celfile.path = celpath, compress = TRUE)
```

Extract gene expression matrix into a standard data frame

```
df_yeast <- data.frame(exprs(yeast_microchip))
```

Preview the top rows of the expression matrix

```
head(df_yeast)
```

	EWE11465_0001_WT1.CEL <dbl>	EWE11465_0002_WT2.CEL <dbl>	EWE11465_0003_WT3.CEL <dbl>	EWE11465_0004
1	75	95	83	
2	4530	4222	4416	
3	74	91	78	
4	4918	4362	4714	
5	111	79	80	
6	66	87	77	

6 rows | 1-5 of 7 columns

Extract Perfect Match (PM) probe signals

```
pm_yeast <- data.frame(affy::pm(yeast_microchip))
```

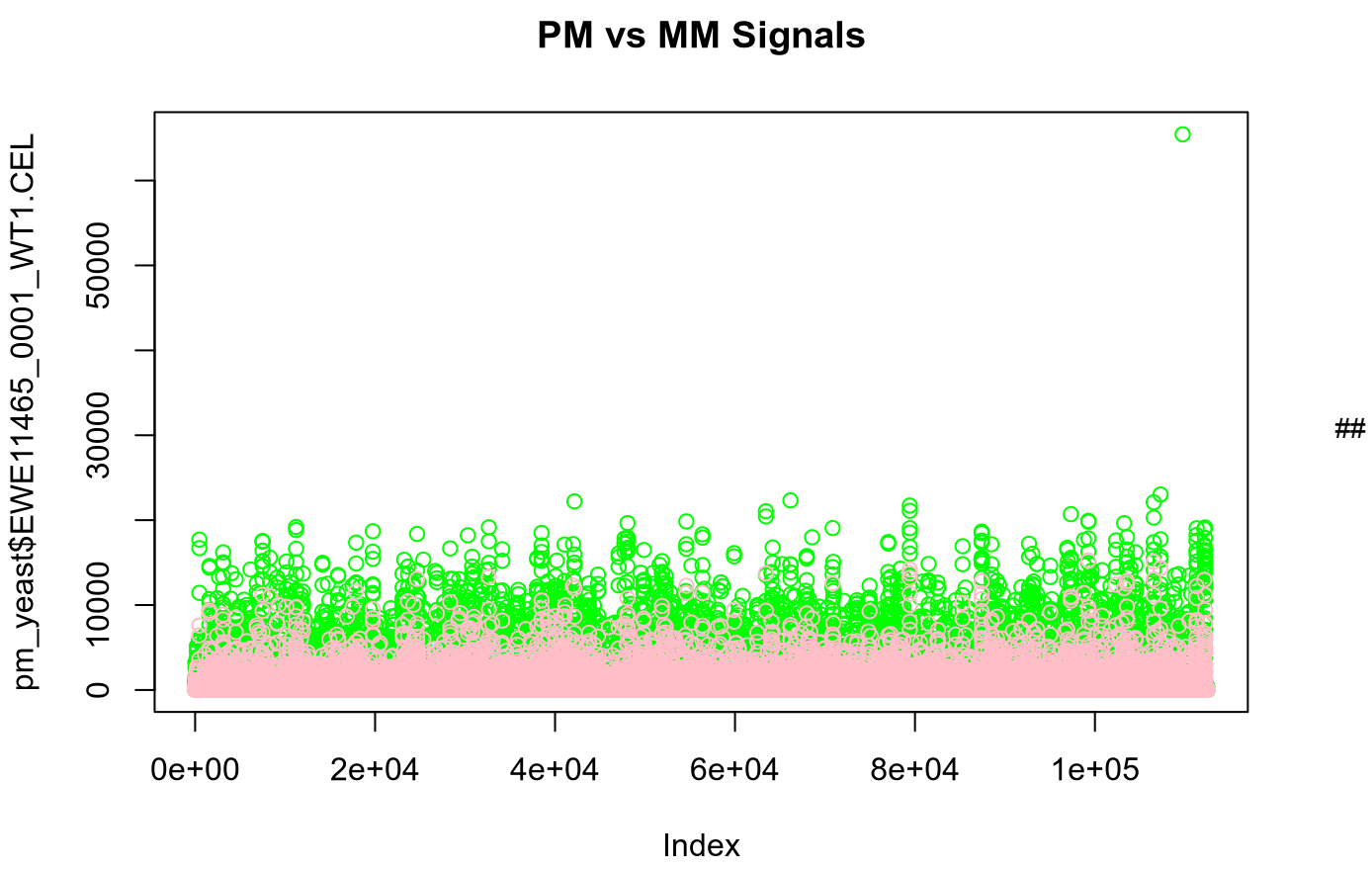
```
##
```

Extract Mismatch (MM) probe signals

```
mm_yeast <- data.frame(affy::mm(yeast_microchip))
```

Scatter plot of PM intensities for the first Wild Type sample

```
plot(pm_yeast$EWE11465_0001_WT1.CEL, col = "green", main = "PM vs MM Signals")
# Overlay MM intensities on the same plot
points(mm_yeast$EWE11465_0001_WT1.CEL, col = "pink")
```



##

Performed log-transformation on PM and MM intensity data

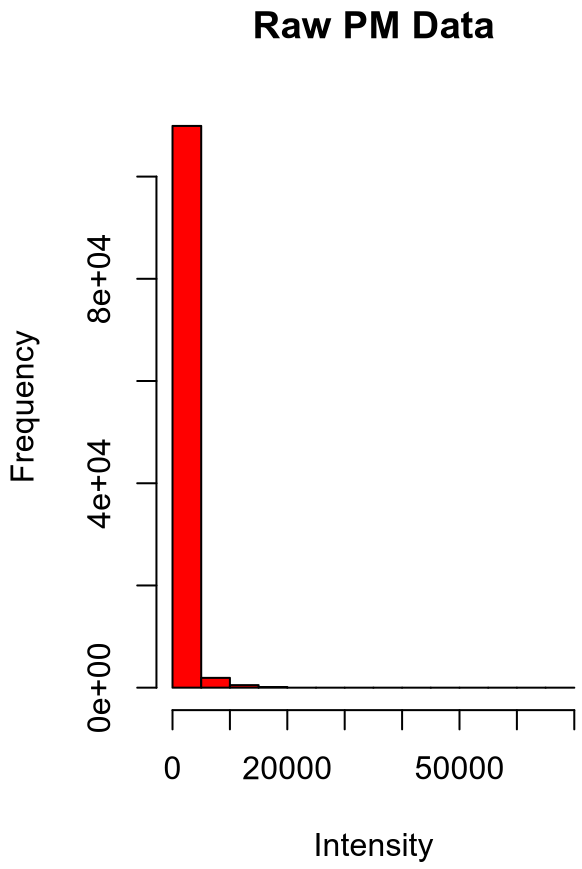
```
pmlog <- log(pm_yeast)
mmlog <- log(mm_yeast)
```

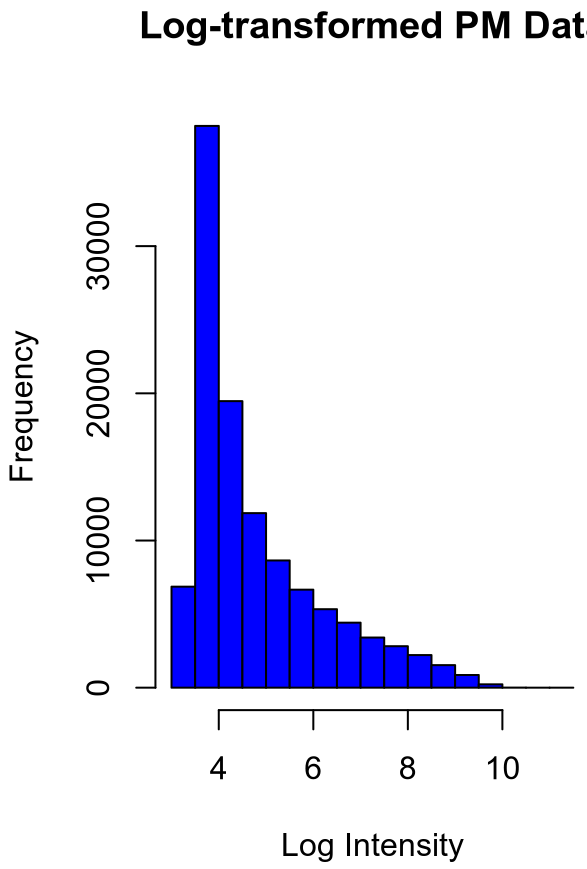
Set up layout for two side-by-side plots

```
par(mfrow = c(1, 2))

# Histogram of raw PM values
hist(pm_yeast$EWE11465_0001_WT1.CEL, main = "Raw PM Data", xlab = "Intensity", col = "re
d")

# Histogram of log-transformed PM values
hist(pmlog$EWE11465_0001_WT1.CEL, main = "Log-transformed PM Data", xlab = "Log Intensit
y", col = "blue")
```

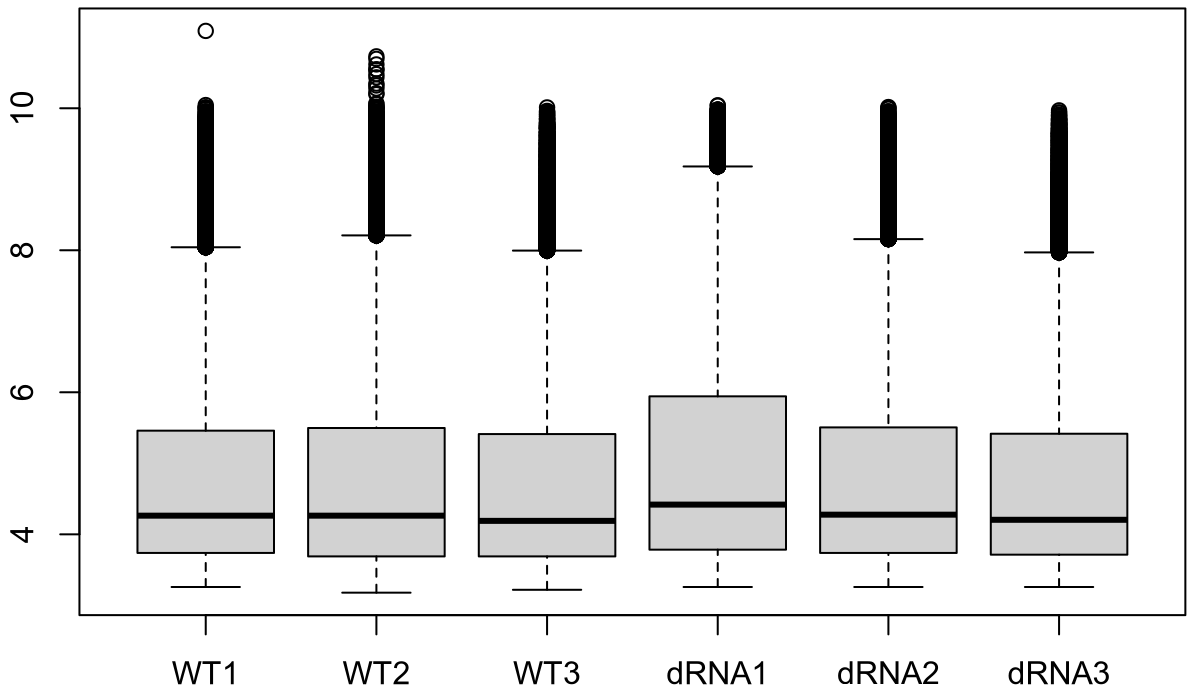




Extract individual sample log-PM vectors

```
wt1_log  <- pmlog$EWE11465_0001_WT1.CEL
wt2_log  <- pmlog$EWE11465_0002_WT2.CEL
wt3_log  <- pmlog$EWE11465_0003_WT3.CEL
drna1_log <- pmlog$EWE11465_0004_drna1.CEL
drna2_log <- pmlog$EWE11465_0005_drna2.CEL
drna3_log <- pmlog$EWE11465_0006_drna3.CEL

# Plot boxplots across samples
boxplot(wt1_log, wt2_log, wt3_log, drna1_log, drna2_log, drna3_log,
        names = c("WT1", "WT2", "WT3", "dRNA1", "dRNA2", "dRNA3"),main = "Log Transforme
d Dataset")
```



##Applied RMA

```
yeast_microchip_rma <- rma(yeast_microchip)
```

```
## Background correcting
## Normalizing
## Calculating Expression
```

##Extraced gene expression matrix into a standard data frame

```
df_rma_yeast <- data.frame(exprs(yeast_microchip_rma))

#the top rows of the expression matrix
head(df_rma_yeast)
```

	EWE11465_0001_WT1.CEL <dbl>	EWE11465_0002_WT2.CEL <dbl>	EWE11465_0003_WT3.CEL <dbl>
1759068_at	3.413791	3.015579	3.437268
1759069_at	9.020832	9.126874	9.048895
1759070_s_at	7.771956	7.623827	7.432955
1759071_s_at	6.148551	5.695259	5.836691

	EWE11465_0001_WT1.CEL <dbl>	EWE11465_0002_WT2.CEL <dbl>	EWE11465_0003_WT3.CEL <dbl>
1759072_s_at	3.670370	3.580104	3.662050
1759073_at	2.925485	3.253104	2.651791

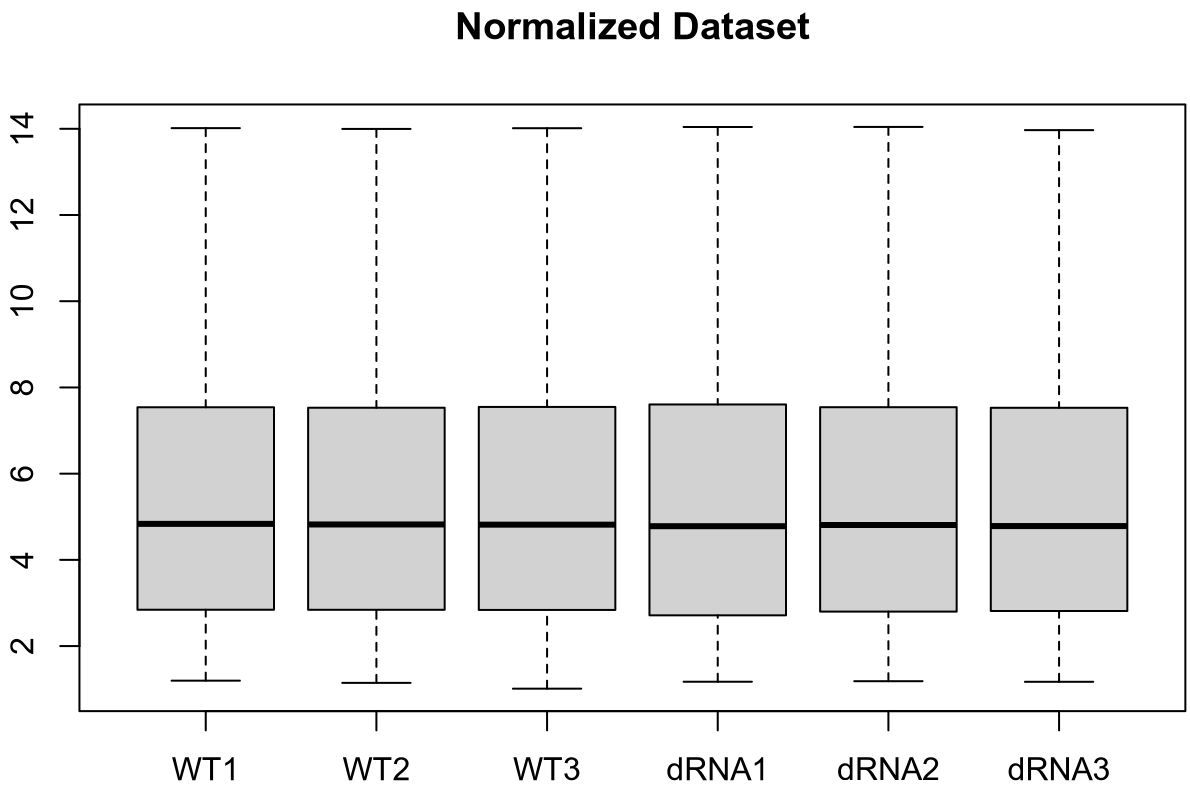
6 rows | 1-4 of 7 columns

##Stored individual sample vectors from the RMA-normalized data frame

```
wt1_norm  <- df_rma_yeast$EWE11465_0001_WT1.CEL
wt2_norm  <- df_rma_yeast$EWE11465_0002_WT2.CEL
wt3_norm  <- df_rma_yeast$EWE11465_0003_WT3.CEL
drna1_norm <- df_rma_yeast$EWE11465_0004_drna1.CEL
drna2_norm <- df_rma_yeast$EWE11465_0005_drna2.CEL
drna3_norm <- df_rma_yeast$EWE11465_0006_drna3.CEL
```

##Boxplot of RMA-normalized expression values

```
boxplot(wt1_norm, wt2_norm, wt3_norm, drna1_norm, drna2_norm, drna3_norm,
        names = c("WT1", "WT2", "WT3", "dRNA1", "dRNA2", "dRNA3"),main = "Normalized Dat
aset")
```



##Prepared raw vectors for comparison

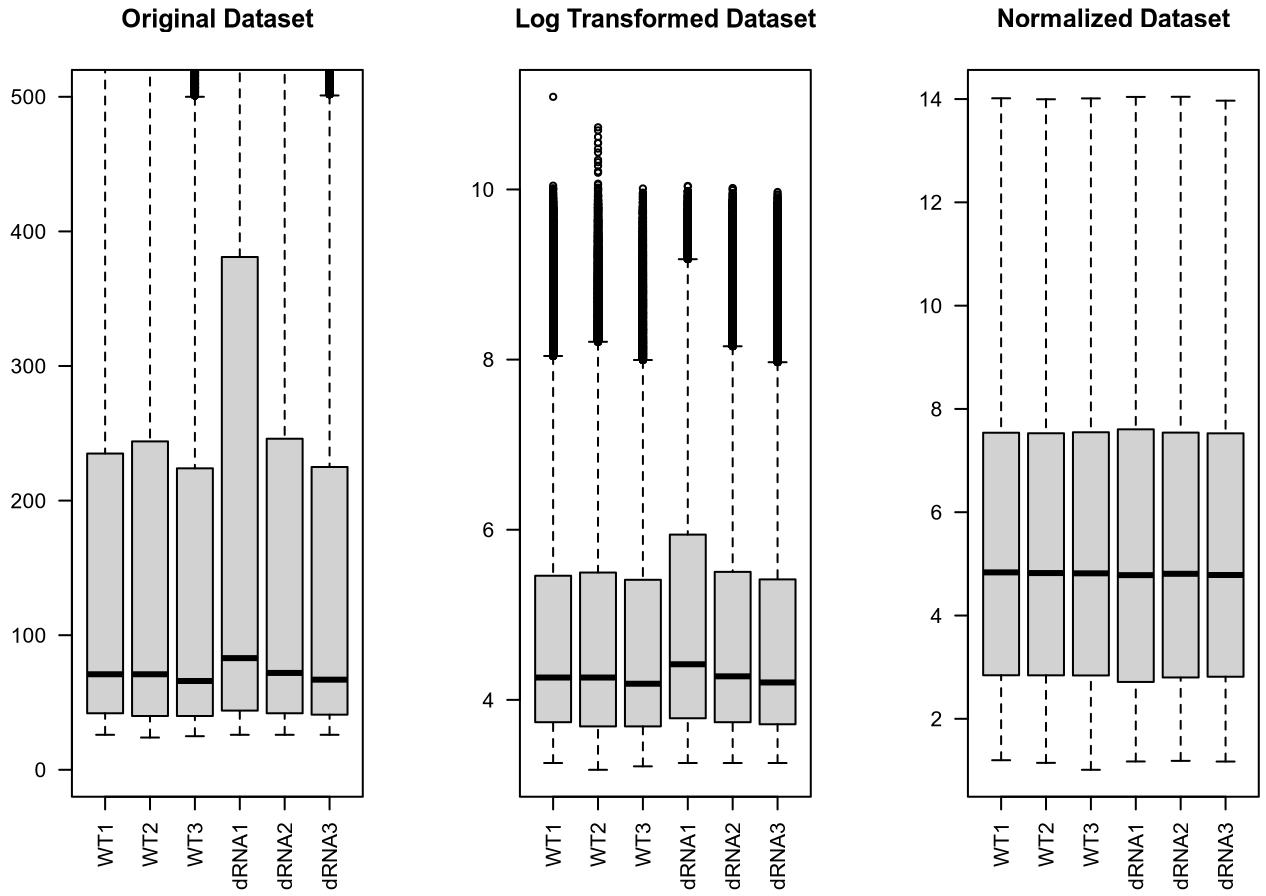
```
wt1_og  <- pm_yeast$EWE11465_0001_WT1.CEL
wt2_og  <- pm_yeast$EWE11465_0002_WT2.CEL
wt3_og  <- pm_yeast$EWE11465_0003_WT3.CEL
drna1_og <- pm_yeast$EWE11465_0004_drna1.CEL
drna2_og <- pm_yeast$EWE11465_0005_drna2.CEL
drna3_og <- pm_yeast$EWE11465_0006_drna3.CEL

# Display three side-by-side boxplots
par(mfrow = c(1, 3))

# 1. Original raw dataset
boxplot(wt1_og, wt2_og, wt3_og, drna1_og, drna2_og, drna3_og,
        names = c("WT1", "WT2", "WT3", "dRNA1", "dRNA2", "dRNA3"),
        ylim = c(0, 500), main = "Original Dataset", las = 2)

# 2. Log-transformed dataset
boxplot(wt1_log, wt2_log, wt3_log, drna1_log, drna2_log, drna3_log,
        names = c("WT1", "WT2", "WT3", "dRNA1", "dRNA2", "dRNA3"),main = "Log Transforme
d Dataset", las = 2)

# 3. RMA-normalized dataset
boxplot(wt1_norm, wt2_norm, wt3_norm, drna1_norm, drna2_norm, drna3_norm,
        names = c("WT1", "WT2", "WT3", "dRNA1", "dRNA2", "dRNA3"),main = "Normalized Dat
aset", las = 2)
```



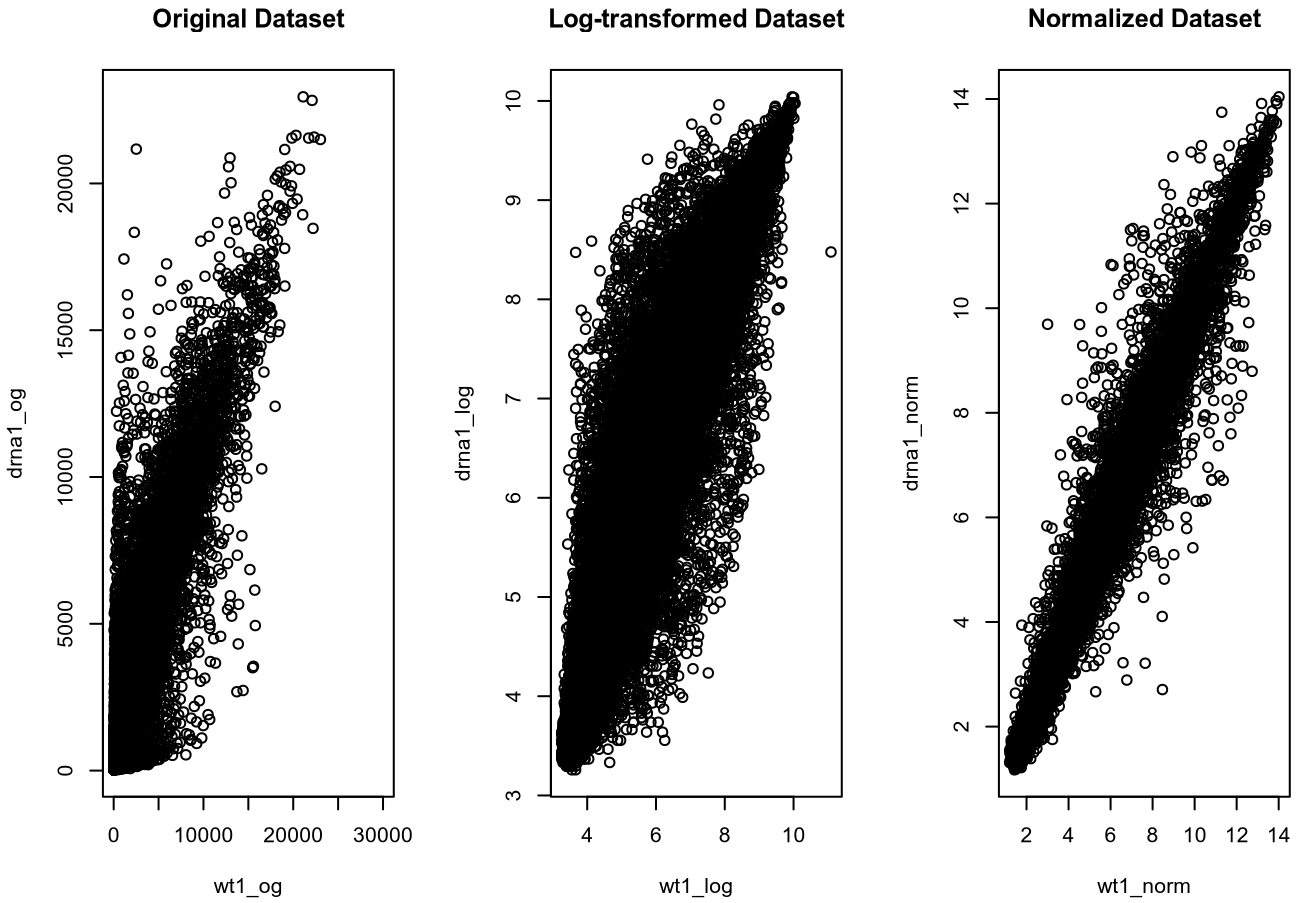
##compared scatter plots

```
par(mfrow = c(1, 3))

# Raw scatter plot
plot(wt1_og, drna1_og, main = "Original Dataset", xlim = c(0, 30000))

# Log-transformed scatter plot
plot(wt1_log, drna1_log, main = "Log-transformed Dataset")

# Normalized scatter plot
plot(wt1_norm, drna1_norm, main = "Normalized Dataset")
```



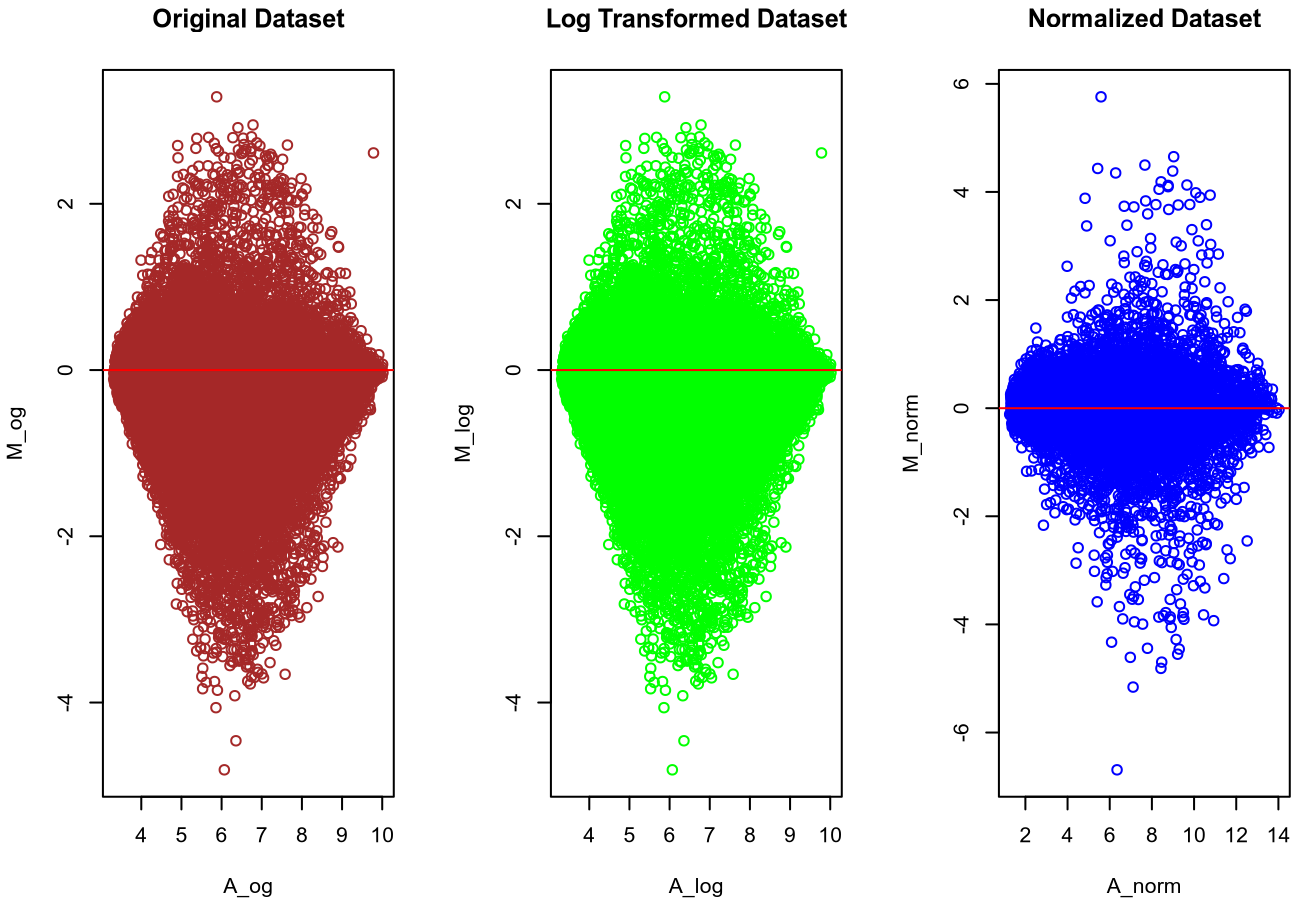
##Generated MA plots

```
par(mfrow = c(1, 3))

# MA Calculation: Raw Data
A_og <- (log(wt1_og) + log(drna1_og)) / 2
M_og <- log(wt1_og / drna1_og)
plot(A_og, M_og, main = "Original Dataset", col='brown')
abline(h = 0, col = "red")

# MA Calculation: Log-Transformed Data
A_log <- (wt1_log + drna1_log) / 2
M_log <- wt1_log - drna1_log
plot(A_log, M_log, main = "Log Transformed Dataset", col='green')
abline(h = 0, col = "red")

# MA Calculation: Normalized Data
A_norm <- (wt1_norm + drna1_norm) / 2
M_norm <- wt1_norm - drna1_norm
plot(A_norm, M_norm, main = "Normalized Dataset", col='blue')
abline(h = 0, col = "red")
```



##This will act as an input for the rest of the part

```
yeast_microchip_rma <- rma(yeast_microchip)
```

```
## Background correcting
## Normalizing
## Calculating Expression
```

```
library(limma)
```

##read the RMA normalized values , printing that we can get to know the exact order

```
pData_yest = pData(yeast_microchip_rma)
pData_yest
```

	sample <int>
EWE11465_0001_WT1.CEL	1
EWE11465_0002_WT2.CEL	2
EWE11465_0003_WT3.CEL	3
EWE11465_0004_drna1.CEL	4
EWE11465_0005_drna2.CEL	5
EWE11465_0006_drna3.CEL	6

6 rows

##We are grouping this,it groups first three as WT group and next three as Mutant, if it is in alternate order , we can write as WT, mutant, WT, mutant like this… #Groups =pData_yeast$Treatnment, order matters here, we can see two levels,if we write WT1 by mistake it will take that as another level, so instead of two here it will become three and WT1 wll not have replicates.

```
Groups = c("WT","WT","WT","Mutant","Mutant","Mutant")
Groups
```

```
## [1] "WT"     "WT"     "WT"     "Mutant" "Mutant" "Mutant"
```

##Purpose: Convert the character vector into a factor ##factors take the parameters in a alphanumeric way,which automatically detects what to compare with whom,,,order does not matter here ##relevel tells what we are going to use as reference..it prevents use of that alphanumeric way,,,and specifically tells what to use as reference ##but use of factor is important to tell how many levels are here.

```
Groups = factor(Groups, levels = c("WT","Mutant"))
Groups = relevel(Groups, ref= "WT")
Groups
```

```
## [1] WT     WT     WT     Mutant Mutant Mutant
## Levels: WT Mutant
```

##We have to create a design matrix, here first three columns, it will take as wt because it has values of 1 for gene 1 and so on and will take the avg..

```
design = model.matrix(~0+Groups)

# design = model.matrix(~0+Groups)..the first 0 ...it  tells to take intercept as 0. No intercept model is also called as means model..without 0,,it will keep the intercept and that will be the mean reference model..there I will get only one column that is mutant. #IN the means model,,if i get 9 in WT and * in Mutant ,,that states the avg intensity,,but in mean reference model the values are in ratio..

design
```

```
##   GroupsWT GroupsMutant
## 1        1            0
## 2        1            0
## 3        1            0
## 4        0            1
## 5        0            1
## 6        0            1
## attr(,"assign")
## [1] 1 1
## attr(,"contrasts")
## attr(,"contrasts")$Groups
## [1] "contr.treatment"
```

##Here the first three columns is 1 for WT,,,so while calculating the avg intensities from gene 1 for wild type it will take the first three column's intensities and put in the formula of B0WT+B1MT,,where it will multiply 0 with the values of next three columns which is effectively mutant types,,,so we will get avg intensity values for WT for gene1. but the avg will be calculated using lmfit…

```
fit_y = lmFit(yeast_microchip_rma, design)
contrast.matrix = makeContrasts(GroupsMutant-GroupsWT, levels = design)
data.fit.con = contrasts.fit(fit_y,contrast.matrix)
fit12 = eBayes(data.fit.con)
contrast.matrix
```

```
##               Contrasts
## Levels         GroupsMutant - GroupsWT
##   GroupsWT                          -1
##   GroupsMutant                       1
```

##makeContrasts is for deifing contrasts, where it is telling to take the differences between mutant and wild type. So it is like specifically defining condition of interest for us.